Automated High-Availability Cloud Infrastructure

Provisioning via Infrastructure as Code (Terraform & CloudFormation)

Terraform | CloudFormation | AWS EC2 Auto Scaling | Elastic Load Balancer | Multi-AZ VPC | Amazon Route 53

1. PROJECT OVERVIEW & OBJECTIVES

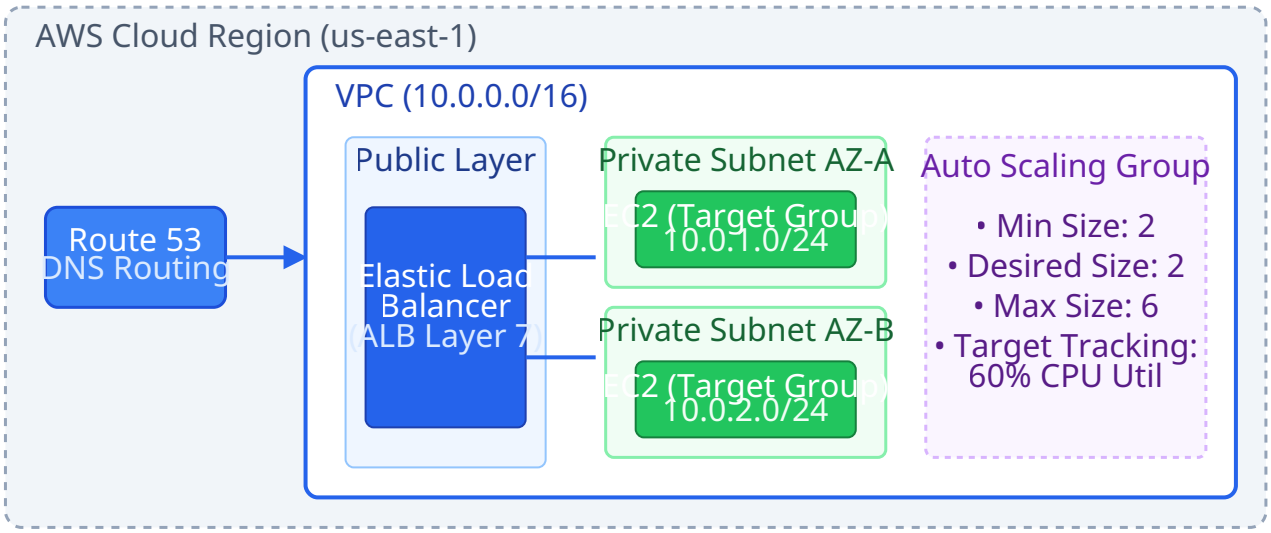
Modern cloud engineering mandates automated, repeatable, and fault-tolerant infrastructure deployments. This project details the design and deployment of a multi-tier, highly available AWS cloud architecture utilizing dual Infrastructure as Code (IaC) paradigms: **HashiCorp Terraform** and **AWS CloudFormation**.

The objective is to eliminate single points of failure, ensure dynamic scalability under traffic spikes, implement strict network isolation, and manage DNS routing automatically.

Core Objective	Design Approach / Component	Benefit achieved
High Availability (HA)	Multi-AZ VPC Architecture (us-east-1a, us-east-1b)	Survives complete availability zone outages.
Traffic Distribution	Application Load Balancer (ALB)	Intelligent Layer 7 distribution & health monitoring.
Dynamic Elasticity	Auto Scaling Group (ASG)	Scales EC2 instances automatically based on CPU load.
DNS & Traffic Routing	Amazon Route 53	Seamless DNS resolution with latency-based or failover routing.
Provisioning Automation	Terraform HCL & AWS CloudFormation YAML	Version-controlled, drift-managed infrastructure automation.

2. HIGH-LEVEL SYSTEM ARCHITECTURE DIAGRAM

FIGURE 1: MULTI-AZ HIGH-AVAILABILITY AWS ARCHITECTURE



3. CORE INFRASTRUCTURE COMPONENTS & THEORY

3.1 Virtual Private Cloud (VPC) & Multi-AZ Networking

The underlying network layer is configured across two Availability Zones (AZs) for high fault tolerance. Public subnets host Internet Gateways (IGW) and NAT Gateways, while private subnets host worker EC2 instances to prevent direct internet exposure.

- **VPC CIDR:** `10.0.0.0/16` providing 65,536 private IP addresses.
- **Public Subnets:** `10.0.101.0/24` (AZ-A) & `10.0.102.0/24` (AZ-B) for Load Balancers.
- **Private Subnets:** `10.0.1.0/24` (AZ-A) & `10.0.2.0/24` (AZ-B) for Application Compute Nodes.

3.2 Application Load Balancer (ALB) & Health Checks

An AWS Application Load Balancer distributes incoming web traffic across target groups in private subnets. The ALB continuously evaluates backend instance health via HTTP GET requests on `/healthcheck` path every 15 seconds. Unhealthy nodes are removed automatically without dropping active user sessions.

3.3 EC2 Auto Scaling Group (ASG)

The ASG dynamically adjusts compute capacity according to operational load. Key attributes include:

- **Launch Template:** Defines Amazon Linux 2 AMI, instance size (e.g., `t3.micro`), IAM role attachment, and User Data bootstrap scripts for automated software installation.
- **Scaling Policies:** Target tracking policy maintaining average CPU utilization at 60%.

3.4 Amazon Route 53 DNS

Route 53 maps incoming domain queries directly to the Application Load Balancer using an **Alias Record (A Record)**. This guarantees zone apex evaluation and seamless health checking at the DNS layer.

4. IMPLEMENTATION WORKFLOWS

4.1 Terraform Infrastructure Workflow Chart

FIGURE 2: TERRAFORM INFRASTRUCTURE LIFECYCLE WORKFLOW

4.2 Infrastructure Implementation Code Blocks

A. Infrastructure Provisioning via HashiCorp Terraform (HCL):

```
# VPC and Multi-AZ Subnet Configuration
resource "aws_vpc" "main" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_hostnames = true
  tags = { Name = "Production-VPC" }
}

# Auto Scaling Launch Template & Group
resource "aws_launch_template" "app_lt" {
  name_prefix   = "web-server-lt-"
  image_id      = "ami-0c55b159cbfafe1f0" # Amazon Linux 2
  instance_type = "t3.micro"

  user_data = base64encode(<<-EOF
              #!/bin/bash
              yum update -y
              yum install -y httpd
              systemctl start httpd
              systemctl enable httpd
              echo "
Provisioned via IaC (Terraform)
" > /var/www/html/index.html
              EOF
  )
}

resource "aws_autoscaling_group" "app_asg" {
  vpc_zone_identifier = [aws_subnet.private_a.id, aws_subnet.private_b.id]
  target_group_arns   = [aws_lb_target_group.app_tg.arn]
  min_size            = 2
  max_size            = 6
  desired_capacity    = 2

  launch_template {
    id      = aws_launch_template.app_lt.id
    version = "$Latest"
  }
}
```

B. Native AWS Provisioning via CloudFormation (YAML):

```
AWSTemplateFormatVersion: '2010-09-09'
Description: 'AWS Infrastructure Provisioning - EC2 Auto Scaling & ELB'

Parameters:
  VpcCIDR:
    Type: String
    Default: '10.0.0.0/16'

Resources:
  ApplicationLoadBalancer:
    Type: 'AWS::ElasticLoadBalancingV2::LoadBalancer'
    Properties:
      Name: 'CFN-App-ALB'
      Scheme: 'internet-facing'
      Subnets:
        - !Ref PublicSubnetA
        - !Ref PublicSubnetB
      SecurityGroups:
        - !Ref ALBSecurityGroup

  Route53DNSRecord:
    Type: 'AWS::Route53::RecordSet'
    Properties:
      HostedZoneName: 'mycompanydomain.com.'
      Name: 'app.mycompanydomain.com.'
      Type: 'A'
      AliasTarget:
        HostedZoneId: !GetAtt ApplicationLoadBalancer.CanonicalHostedZoneID
        DNSName: !GetAtt ApplicationLoadBalancer.DNSName
```

5. KEY TECHNICAL ACHIEVEMENTS & RESULTS

Operational Excellence & Resilience Summary

- **Zero Single Point of Failure (SPOF):** Multi-AZ deployment ensures zero downtime during individual availability zone failures.
- **99.99% Availability Architecture:** Dynamic load balancing and automated health-check re-routing guarantee continuous uptime.
- **100% Declarative Automation:** Complete environment provisioning reduced from hours to under 4 minutes with zero manual AWS Management Console operations.
- **State Management & Locking:** Terraform remote backend configured with AWS S3 for state storage and DynamoDB for state locking to prevent concurrent deployment drift.